Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 5

Subject	DBMS Lab
Assignment No.	5
Topic	Cursor code using control structures and exception handling.
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	24/03/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To study and implement cursors, control structures, and exception handling in SQL.

Objectives:

- To understand the concept of cursors in SQL.
- To implement control structures like loops and conditions.
- To handle exceptions using SQL handlers.
- To perform row-by-row data processing using cursors.

Theory: Cursor, Control Structures and Exception Handling

A cursor is a database object used to retrieve data row by row from a result set. It is useful when we need to process each record individually rather than working with the entire dataset at once.

Control structures such as IF conditions, loops, and labels are used within procedures to control the flow of execution. These structures help in implementing logical decision-making and repetitive operations.

Exception handling is used to manage runtime errors in SQL. Handlers such as CONTINUE and EXIT are used to handle exceptions like NOT FOUND or SQL errors, ensuring smooth execution of database operations.

These concepts are essential for performing complex and dynamic database operations efficiently.

Case Study: Airline Management System

SQL Queries (Cursor, Control Structures & Exception Handling):

Part A: Cursor with Control Structures

---

1. Cursor Example:

Display all passenger names using cursor.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE DisplayPassengers()
   -> BEGIN
   ->     DECLARE done INT DEFAULT 0;
   ->     DECLARE pname_var VARCHAR(50);
   ->     DECLARE cur CURSOR FOR SELECT pname FROM passenger;
   ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   ->
   ->     OPEN cur;
   ->
   ->     read_loop: LOOP
   ->         FETCH cur INTO pname_var;
   ->         IF done = 1 THEN
   ->             LEAVE read_loop;
   ->         END IF;
   ->         SELECT pname_var;
   ->     END LOOP;
   ->
   ->     CLOSE cur;
   -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

---

2. Call Procedure

```
mysql> CALL DisplayPassengers();
+--------------+
| pname_var    |
+--------------+
| Rahul Sharma |
| Neha Verma   |
| Amit Patil   |
+--------------+
3 rows in set (0.00 sec)
```

---

3. Cursor with Condition:
Display flights with price > 5000.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE HighPriceFlights()
   -> BEGIN
   ->     DECLARE done INT DEFAULT 0;
   ->     DECLARE fname VARCHAR(20);
   ->     DECLARE fprice DECIMAL(10,2);
   ->     DECLARE cur CURSOR FOR SELECT flight_name, price FROM flight;
   ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   ->
   ->     OPEN cur;
   ->
   ->     read_loop: LOOP
   ->         FETCH cur INTO fname, fprice;
   ->         IF done = 1 THEN
   ->             LEAVE read_loop;
   ->         END IF;
   ->
   ->         IF fprice > 5000 THEN
   ->             SELECT fname, fprice;
   ->         END IF;
   ->
   ->     END LOOP;
   ->
   ->     CLOSE cur;
   -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

---

4. Call Procedure

```
mysql> CALL HighPriceFlights();
+--------+--------+
| fname  | fprice |
+--------+--------+
| AI101  | 5500   |
| EM404  | 15000  |
+--------+--------+
2 rows in set (0.00 sec)
```

Part B: Exception Handling

5. Procedure with Exception Handling:

Handle division by zero.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE DivideNumbers(IN a INT, IN b INT)
   -> BEGIN
   ->     DECLARE result INT;
   ->     DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
   ->     SELECT 'Error occurred (possible division by zero)' AS message;
   ->
   ->     SET result = a / b;
   ->     SELECT result;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

6. Call Procedure (Valid Case)

```
mysql> CALL DivideNumbers(10,2);
+--------+
| result |
+--------+
| 5      |
+--------+
1 row in set (0.00 sec)
```

7. Call Procedure (Exception Case)

```
mysql> CALL DivideNumbers(10,0);
+-------------------------------------------+
| message                                   |
+-------------------------------------------+
| Error occurred (possible division by zero)|
+-------------------------------------------+
1 row in set (0.00 sec)
```

Case Study: Civic Voice Interface System

SQL Queries (Cursor, Control Structures & Exception Handling):

Part A: Cursor with Control Structures

---

1. Cursor Example:

Display all user names.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE DisplayUsers()
   -> BEGIN
   ->     DECLARE done INT DEFAULT 0;
   ->     DECLARE uname VARCHAR(50);
   ->     DECLARE cur CURSOR FOR SELECT name FROM user;
   ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   ->
   ->     OPEN cur;
   ->
   ->     read_loop: LOOP
   ->         FETCH cur INTO uname;
   ->         IF done = 1 THEN
   ->             LEAVE read_loop;
   ->         END IF;
   ->         SELECT uname;
   ->     END LOOP;
   ->
   ->     CLOSE cur;
   -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

---

2. Call Procedure

```
mysql> CALL DisplayUsers();
+--------------+
| uname        |
+--------------+
| Rahul Sharma |
| Neha Verma   |
| Amit Patil   |
+--------------+
3 rows in set (0.00 sec)
```

---

3. Cursor with Condition:

Display only pending complaints.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE PendingComplaints()
    -> BEGIN
    ->     DECLARE done INT DEFAULT 0;
    ->     DECLARE issue_var VARCHAR(100);
    ->     DECLARE stat VARCHAR(20);
    ->     DECLARE cur CURSOR FOR SELECT issue, status FROM complaint;
    ->     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->
    ->     OPEN cur;
    ->
    ->     read_loop: LOOP
    ->         FETCH cur INTO issue_var, stat;
    ->         IF done = 1 THEN
    ->             LEAVE read_loop;
    ->         END IF;
    ->
    ->         IF stat = 'Pending' THEN
    ->             SELECT issue_var, stat;
    ->         END IF;
    ->
    ->     END LOOP;
    ->
    ->     CLOSE cur;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

---

4. Call Procedure

```
mysql> CALL PendingComplaints();
+----------------+---------+
| issue_var      | stat    |
+----------------+---------+
| Road damage    | Pending |
| Garbage issue  | Pending |
+----------------+---------+
2 rows in set (0.00 sec)
```

Part B: Exception Handling

5. Procedure with Exception Handling:

Handle invalid user ID.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetUserById(IN uid INT)
   -> BEGIN
   ->     DECLARE CONTINUE HANDLER FOR NOT FOUND
   ->     SELECT 'User not found' AS message;
   ->
   ->     SELECT * FROM user WHERE user_id = uid;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

6. Call Procedure (Valid Case)

```
mysql> CALL GetUserById(1);
+---------+--------------+------------------------+--------------+----------+
| user_id | name         | email                  | phone_number | language |
+---------+--------------+------------------------+--------------+----------+
| 1       | Rahul Sharma | rahul@email.com        | 9876543210   | English  |
+---------+--------------+------------------------+--------------+----------+
1 row in set (0.00 sec)
```

7. Call Procedure (Exception Case)

```
mysql> CALL GetUserById(10);
+-------------------+
| message           |
+-------------------+
| User not found    |
+-------------------+
1 row in set (0.00 sec)
```

Conclusion:

In this assignment, cursors, control structures, and exception handling were successfully implemented. These features allow efficient row-by-row processing, logical control of execution, and proper handling of errors. Thus, they play a crucial role in advanced database programming.